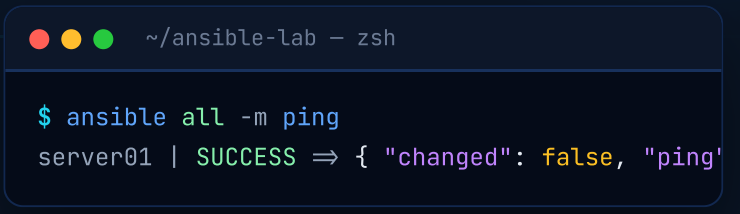
Ansible

Configuration Management and Automation

Linux Administration and DevOps Training — Beginner to Intermediate



A complete classroom and self-study course

11 Modules | 66 Slides | Hands-on Labs Included

Learning Objectives

What you will master in this course

01

Ansible Fundamentals

Understand what Ansible is, why it exists, and how it solves real Linux administration problems.

02

Architecture & Components

Master control nodes, managed nodes, inventory, modules, playbooks, and SSH-based execution.

03

Ad-hoc Commands

Run one-off automation tasks across many servers using modules like ping, command, dnf, and service.

04

Playbook Authoring

Write YAML playbooks that install packages, manage files, services, users, and directories.

05

Variables, Facts, Loops & Handlers

Use dynamic data, system facts, loops for repeated tasks, and handlers for event-driven restarts.

06

Vault, Roles & Galaxy

Protect secrets with Ansible Vault and structure reusable automation with roles and Ansible Galaxy.

"By the end of this course, you will automate a 3-server Linux environment end-to-end."

What is Ansible?

A simple, powerful, agentless automation tool

- **Ansible** is an open-source automation tool used to configure systems, deploy applications, and orchestrate IT workflows.
- It was created by **Michael DeHaan** in **2012** and is now maintained by **Red Hat** as a top-level community project.
- Written in **Python**, but you do NOT need to write Python to use it — you write simple **YAML** files.
- **Configuration Management:** keeping servers in a known, consistent, repeatable state (packages, services, files, users).
- **Automation:** replacing manual, repetitive SSH sessions with code that runs across many servers at once.
- For Linux administrators, Ansible means: type the desired state **ONCE**, apply it to **ONE** server or **ONE THOUSAND** servers identically.

In Simple Words

"Ansible is a tool that lets you log into many Linux servers at the same time, run tasks on all of them with a single command, and make sure each server ends up in exactly the state you asked for — without installing any agent software on those servers."

demo — bash

```
$ ansible all -m ping
web01 | SUCCESS ⇒ { "ping": "pong" }
web02 | SUCCESS ⇒ { "ping": "pong" }
db01  | SUCCESS ⇒ { "ping": "pong" }
```

Why Do We Need Ansible?

The cost of manual Linux administration

Problems with Manual Administration

01 Managing multiple servers manually — SSH into each box one at a time.

02 Repeating the same commands on every server, every time.

03 **Configuration drift** — servers slowly become different from each other.

04 Time-consuming — a 5-minute task becomes 50 minutes across 10 servers.

05 Human errors — typos, forgotten steps, wrong order of operations.

06 No audit trail — "who changed what, and when?" is impossible to answer.

How Ansible Solves These

- One command or one playbook runs against dozens or hundreds of servers **in parallel**.
- Desired state is written **once** in YAML and applied identically everywhere.
- **Idempotency** — running the same playbook 100 times produces the same result as running it once.
- Tasks are documented as code: version-controlled, reviewable, repeatable.
- Free time is reclaimed for engineering work instead of repetitive operations.

"Stop doing the same thing 50 times. Write it once. Run it everywhere."

What is Ansible Used For?

Real-world use cases in modern IT operations

Configuration Management

Enforce consistent server state: packages, files, services, users.

```
Example: ensure httpd is installed and running on all 50 web servers.
```

Application Deployment

Push code, configs, and assets; restart services in the right order.

```
Example: deploy a new app version to web servers, then reload Apache.
```

Package Installation

Install, update, or remove RPM/DEB packages across hosts.

```
Example: ansible webservers -b -m dnf -a "name=httpd state=present"
```

Service Management

Start, stop, restart, and enable services on boot.

```
Example: ensure nginx is started and enabled on every web node.
```

User & Group Management

Create users, set SSH keys, manage sudoers, remove departed staff.

```
Example: create user 'jooli' with SSH key on all dev servers.
```

Cloud Infrastructure Automation

Provision VMs, networks, and storage on AWS, Azure, GCP, OpenStack.

```
Example: spin up 3 EC2 instances and configure them as a web cluster.
```

Main Functions of Ansible

Core capabilities that make Ansible powerful

Orchestration
Coordinate multi-tier deployments: run tasks on web tier, then db tier, in the right order.

Remote Execution
Run any Linux command on many hosts at once, from a single control node.

Desired State Enforcement
Declare the end state you want; Ansible figures out how to get there.

Idempotent Operations
Run a playbook 100 times — the system ends up in the same state as running it once.

Agentless Push Model
No daemon installed on managed nodes; Ansible pushes changes over SSH.

Declarative YAML Playbooks
Human-readable automation written in YAML, easy to read, review, and version.

Module Ecosystem
Hundreds of built-in modules: files, packages, services, cloud, containers, network.

Reusable Roles & Collections
Package automation into roles, share via Ansible Galaxy, reuse across teams.

Ansible = Push + SSH + YAML + Modules + Idempotency

Features and Advantages of Ansible

Strengths — and honest limitations

Key Features & Advantages

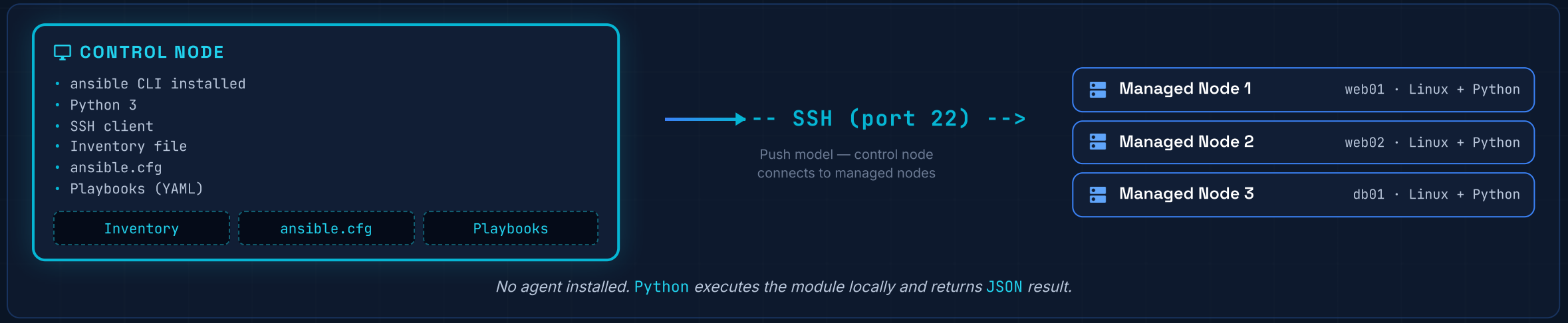
- **Agentless** — nothing to install on managed Linux nodes; uses existing SSH.
- **SSH-based** — communicates over standard SSH; no extra ports or daemons to secure.
- **YAML Playbooks** — declarative, human-readable, no programming experience needed.
- **Idempotent** — safe to run repeatedly; only changes what is not already in the desired state.
- **Easy to learn** — a Linux admin can be productive in hours, not weeks.
- **Open-source** — free to use, community-driven, supported commercially by Red Hat.
- **Cross-platform** — Linux, Windows, network devices, cloud APIs, containers.
- **Extensible** — write your own modules in Python if the built-ins are not enough.

⚠ Limitations to Be Aware Of

- **Linux/Unix control node only** — the machine running Ansible must run Linux/macOS (not Windows as control node).
- **Python required** on managed Linux nodes for most modules (usually pre-installed).
- **Push model** — the control node must be able to reach every managed host over SSH.
- **Not ideal for real-time event-driven automation** — it runs on demand, not as a continuous daemon.
- Large, very complex orchestration can become hard to manage without roles and structure.
- Performance at very large scale (10,000+ hosts) requires careful tuning of `forks` and SSH multiplexing.

Ansible Architecture

Control node, managed nodes, and the execution flow



EXECUTION FLOW

01 Admin runs `ansible` or `ansible-playbook` on control node

02 Ansible reads inventory + ansible.cfg

03 Ansible parses the playbook / ad-hoc command

04 Ansible opens SSH to each managed host in parallel (controlled by `forks`)

05 Ansible copies the module Python code to the managed node (temp)

06 Module executes on the managed node and returns JSON result

07 Ansible removes the temp module and reports success/failure back

Control Node vs Managed Nodes

Two roles in every Ansible deployment

	Control Node	Managed Node
ROLE	The machine that runs Ansible	The server being configured by Ansible
SOFTWARE	`ansible`, `ansible-core`, Python 3, SSH client	Only Python 3 (for most modules) + SSH server
AGENT?	N/A (this IS the orchestrator)	No agent. Ansible uses existing SSH.
OS	Linux (RHEL, AlmaLinux, Rocky, Ubuntu, Debian) or macOS	Linux, Windows, network devices, cloud APIs
INVENTORY	Listed as the source of automation	Listed in the inventory file as a target
EXAMPLE	`admin-lab.example.com` (your laptop or jump host)	`web01`, `web02`, `db01` (production servers)

Practical Example

```
# On the control node (admin-lab):
$ ansible webservers -i inventory -m ping

# This pings web01 and web02 (managed nodes) over SSH.
# No software was installed on web01 or web02 beforehand.
# Ansible SSH'd in, ran the ping module via Python, and returned the result.
```

What is an Ansible Inventory?

How Ansible knows which servers to manage

- An **inventory** is a list of managed hosts that Ansible connects to and configures.
- It is the **single source of truth** that tells Ansible: "these are the servers I want to manage."
- Without an inventory, Ansible has no idea which machines to SSH into.
- An inventory can be a plain text file (**static inventory**) or generated dynamically from a cloud provider (**dynamic inventory**).
- Hosts in an inventory can be organized into **GROUPS** so you can target subsets of servers (e.g. "all webservers", "all dbservers").
- A single host can belong to **multiple groups** at the same time.
- Default location: `/etc/ansible/hosts` — but you almost always specify your own with the `-i` flag.

Why Inventory Is Required

"Ansible never guesses which servers to touch. Every `ansible` and `ansible-playbook` command reads an inventory to know:

- which hosts to contact,
- how to group them,
- which connection settings to use.

A typical classroom inventory file is just a few lines of text — one IP or hostname per line, grouped under `[group_name]` headers."

KEY TAKEAWAY — Inventory = the phone book Ansible uses to call your servers.

Static Inventory Example

A simple INI-style inventory file

inventory — ~/ansible-lab/inventory

```
# Static inventory file - INI format
# Comments start with #

[webservers]
192.168.1.10
192.168.1.11

[dbservers]
192.168.1.20

[labs:children]
webservers
dbservers
```

Section Breakdown

01 `[webservers]` — a group named "webservers" containing two hosts.

02 `192.168.1.10 / .11` — the IP addresses of the two web servers.

03 `[dbservers]` — a separate group for database servers (one host here).

04 `[labs:children]` — a GROUP OF GROUPS; "labs" contains both webservers and dbservers.

targeting groups

```
$ ansible webservers -m ping   # only the 2 web servers
$ ansible dbservers  -m ping   # only the db server
$ ansible labs       -m ping   # all 3 via parent group
$ ansible all        -m ping   # every host in the inventory
```

Inventory File Location

Default paths and the -i flag

Where Ansible Looks for Inventory

- **Default path:** `/etc/ansible/hosts` (system-wide, rarely used in modern setups).
- **Custom path via `-i` flag:** `ansible all -i ./inventory -m ping`
- **Multiple files at once:** `ansible all -i ./inventory -i ./extra-hosts -m ping`
- **A directory of inventory files:** `ansible all -i ./inventory-dir/ -m ping` (Ansible merges all files inside).
- `ansible.cfg` can set a default inventory path so you never need `-i`.
- **Best practice:** keep inventory in your project folder under version control (git), never in `/etc/ansible/`.

list inventory

```
$ ansible-inventory -i inventory --list
{
    "_meta": { "hostvars": {} },
    "all": {
        "children": [
            "dbservers",
            "labs",
            "ungrouped",
            "webservers"
        ]
    },
    "dbservers": { "hosts": ["192.168.1.20"] },
    "webservers": { "hosts": ["192.168.1.10","192.168.1.11"] }
}
```

graph view

```
$ ansible-inventory -i inventory --graph
@all:
  |--@labs:
  |  |--@webservers:
  |  |  |--192.168.1.10
  |  |  |--192.168.1.11
  |  |--@dbservers:
```

What is ansible.cfg?

The configuration file that tunes Ansible's behavior

- `ansible.cfg` is the central configuration file for Ansible — it controls **HOW** Ansible behaves.
- It is **NOT** the inventory (which lists **WHAT** to manage); it is settings like default inventory path, remote user, SSH behavior, privilege escalation, parallelism.
- Most classroom/laptop setups keep `ansible.cfg` in the project directory right next to the inventory and playbooks.
- **Ansible searches for ansible.cfg in this exact order** (first match wins):
  1. `$ANSIBLE_CONFIG` environment variable (if set, points to a file)
  2. `./ansible.cfg` in the current directory
  3. `~/.ansible.cfg` in the user's home directory
  4. `/etc/ansible/ansible.cfg` (system-wide default)
- A custom `ansible.cfg` in your project folder overrides the system one — this is what you almost always want.
- Without an `ansible.cfg`, Ansible falls back to built-in defaults.

ansible.cfg vs inventory

File	What it controls
`ansible.cfg`	Controls **BEHAVIOR** (how Ansible runs)
`inventory`	Controls **TARGETS** (which hosts to manage)

"ansible.cfg = the rules. inventory = the targets.
They are different files for different jobs."

Tip: run `ansible-config list` to see every available setting and its current value.

Important ansible.cfg Settings

A practical sample configuration

ansible.cfg — ~/ansible-lab/ansible.cfg

```
[defaults]
inventory      = ./inventory
remote_user    = devops
```

Setting	Meaning
inventory	Path to your default inventory file (so you can skip -i)
remote_user	SSH user Ansible logs in as on managed nodes
host_key_checking	False = do not prompt for SSH host key acceptance (handy in labs)
become	True = enable privilege escalation by default for tasks that need root
become_method	sudo (or su, pbrun, etc.) — how to escalate
forks	Number of hosts Ansible configures in parallel (default 5)
timeout	SSH connection timeout in seconds

NOTE — become=True here sets a DEFAULT; you can still override per-task or per-play in a playbook.

What are Ad-hoc Commands?

Quick, one-off automation without writing a playbook

- An **ad-hoc command** is a single Ansible command run from the shell to perform **ONE** task on one or more hosts.
- Use them when you need to do something quickly and you do not need to save the task as repeatable code.
- Typical use cases: *"is every server reachable?"*, *"what's the disk space on the db tier?"*, *"install this package on the web tier right now"*.
- Ad-hoc commands use the **same modules** that playbooks use — they are NOT a separate tool.
- They are perfect for **exploration, troubleshooting, and quick operational fixes**.
- Anything you can do ad-hoc, you can also do in a playbook — and once a task becomes repeatable, you **SHOULD** move it into a playbook.

Ad-hoc vs Playbook

Ad-hoc	One command, one task, not saved, run once
Playbook	A YAML file of many tasks, version-controlled, repeatable

ad-hoc — check uptime now

```
# Ad-hoc: check uptime now
$ ansible all -m command -a "upt
```

same task in a playbook

```
# Same task in a playbook:
- name: Check uptime
  ansible.builtin.command:
    cmd: uptime
```

Ad-hoc Command Syntax

Anatomy of an ansible command

general syntax

```
ansible  [host-pattern]  -i [inventory]  -m [module]  -a "[arguments]"
```

ansible

The Ansible CLI binary itself.

[host-pattern]

Which hosts to target. Examples: `all`, `webservers`, `web01`, `'webservers:!db01'`

-i inventory

Path to the inventory file. Optional if `ansible.cfg` sets one.

-m module

Which Ansible module to run: `ping`, `command`, `dnf`, `service`, `file`, `copy`...

-a "args"

Arguments to the module, usually as `key=value` pairs.

REAL EXAMPLE — CHECK UPTIME ON WEBSERVERS

```
$ ansible webservers -i inventory -m command -a "uptime"
```

WITH PRIVILEGE ESCALATION — INSTALL HTTPD

```
$ ansible webservers -i inventory -b -m dnf -a "name=httpd state=present"
```

Understanding Ansible Modules

The reusable building blocks of Ansible

- A **module** is a small, reusable unit of code that performs a specific task on a managed node.
- Modules are written in **Python** (most of them) and shipped with Ansible or via collections.
- When you run a module, Ansible pushes its code over SSH to the managed node, runs it, and returns the result as **JSON**.
- Modules are **idempotent**: running them multiple times produces the same final state; they only report *"changed"* when they actually did something.
- Every module has a name. Modern Ansible uses **fully qualified collection names (FQCN)** like `ansible.builtin.copy`, `ansible.builtin.service`.
- There are **hundreds of modules**: files, packages, services, users, cloud, containers, network, databases...

Common Built-in Modules

Module	What it does
`ansible.builtin.ping`	Tests SSH + Python on the host
`ansible.builtin.command`	Runs a shell command (no pipes)
`ansible.builtin.shell`	Runs a command via /bin/sh (pipes, redirects)
`ansible.builtin.dnf`	RPM packages on RHEL/AlmaLinux/Rocky
`ansible.builtin.apt`	DEB packages on Debian/Ubuntu
`ansible.builtin.service`	Start/stop/enable services
`ansible.builtin.file`	Create files/dirs, set permissions
`ansible.builtin.copy`	Copy a file from control → managed node
`ansible.builtin.user`	Create/remove Linux users
`ansible.builtin.debug`	Print values during playbook

Ad-hoc Practical Examples

Seven common commands you will use every day

1 Test connectivity

```
$ ansible all -m ping
```

RESULT: Returns SUCCESS + pong if SSH + Python work. No changes made.

2 Check uptime

```
$ ansible all -m command -a "uptime"
```

RESULT: Prints load average and uptime on every host.

3 Check disk space

```
$ ansible all -m command -a "df -h"
```

RESULT: Prints filesystem usage in human-readable form.

4 Check memory

```
$ ansible all -m command -a "free -h"
```

RESULT: Prints RAM and swap usage per host.

5 Install a package

```
$ ansible webservers -b -m dnf -a "name=httpd state=present"
```

RESULT: Installs httpd on webservers using sudo (-b). Reports changed if newly installed.

6 Start a service

```
$ ansible webservers -b -m service -a
```

RESULT: Starts httpd. Reports changed only if not already running.

7 Check the date

```
$ ansible all -m command -a "date"
```

RESULT: Prints system date/timezone — useful for spotting NTP drift.

TIP — add `-o` (one-line) for compact output across many hosts: `ansible all -m command -a "hostname" -o`

Important Ad-hoc Command Options

Flags you will use constantly

Flag	Name	Example	Meaning
`-i`	inventory	`-i ./inventory`	Use this inventory file (overrides ansible.cfg)
`-m`	module	`-m ping`	Which Ansible module to execute
`-a`	args	`-a "name=httpd state=present"`	Arguments passed to the module
`-b`	become	`-b`	Run with privilege escalation (sudo by default)
`-k`	ask-pass	`-k`	Prompt for SSH password (when no SSH keys)
`-K`	ask-become-pass	`-K`	Prompt for the sudo password
`-u`	user	`-u devops`	SSH in as this user (overrides remote_user)

COMBINED REAL-WORLD EXAMPLE

```
$ ansible webservers -i inventory -u devops -k -b -K -m dnf -a "name=httpd state=present"
```

"Log in as 'devops' (prompt for SSH password), escalate to root (prompt for sudo password), install httpd on every host in the webservers group."

PRO TIP – In production you should use SSH keys and passwordless sudo so `-k` and `-K` disappear. They are common in classroom/lab setups only.

Ad-hoc Commands Practical Lab

Scenario — manage 3 remote Linux servers

Scenario

"You are the Linux administrator for a small environment with three **AlmaLinux 9** servers:

```
web01   192.168.1.10
web02   192.168.1.11
db01    192.168.1.20
```

Your manager asks you to:

- verify all three servers are reachable,
- check disk space and uptime on each,
- install the Apache HTTP server on the two web servers,
- start and enable httpd on the web servers.

*You will do all of this from your control node **WITHOUT** logging in to any server individually — using only Ansible ad-hoc commands.*

inventory file

```
# inventory file
[webservers]
192.168.1.10
192.168.1.11
[dbservers]
192.168.1.20
```

Tasks to Run

01 Connectivity check

```
$ ansible all -i inventory -m ping
```

02 Uptime across all hosts

```
$ ansible all -i inventory -m command -a "uptime"
```

03 Disk space across all hosts

```
$ ansible all -i inventory -m command -a "df -h"
```

04 Install Apache on web servers (sudo)

```
$ ansible webservers -i inventory -b -m dnf -a "name=httpd state=present"
```

05 Start and enable httpd

```
$ ansible webservers -i inventory -b -m service -a "name=httpd state=started enab
```

"Result: 5 commands, 3 servers fully verified and Apache running on the web tier. Total time: under 30 seconds."

What is an Ansible Playbook?

Repeatable automation written in YAML

- A **playbook** is a YAML file that describes a list of **tasks** Ansible should run on a set of hosts, in order.
- It is the way you turn one-off ad-hoc commands into **repeatable, version-controlled automation**.
- A playbook contains one or more **PLAYS**. Each play targets a host group and contains a list of **TASKS**.
- Each task calls a **module** (`copy`, `dnf`, `service`, `file`, etc.) with arguments.
- Playbooks are **idempotent** — run them 100 times, the system ends in the same state.
- They are **human-readable**: a sysadmin who has never used Ansible can usually read a playbook and understand what it does.

Ad-hoc vs Playbook — when to use which

Ad-hoc	Run-once tasks, exploration, debugging
Playbook	Repeatable processes, deployments, configuration

same task both ways

```
# Ad-hoc (one-off)
$ ansible webservers -b -m dnf -a "name=httpd state=present"

# Playbook (repeatable, saved as install_httpd.yml)
- name: Install Apache
  hosts: webservers
  become: true
  tasks:
    - name: Install httpd
      ansible.builtin.dnf:
        name: httpd
        state: present
```

"If you type an ad-hoc command more than once — turn it into a playbook."

Playbook Structure

Anatomy of a YAML playbook

site.yml

```yaml
---                                    # YAML document start
- name: Install and start Apache
  hosts: webservers
  become: true
  tasks:
    - name: Install httpd package
      ansible.builtin.dnf:
        name: httpd
        state: present
    - name: Start and enable httpd
      ansible.builtin.service:
        name: httpd
        state: started
        enabled: true
```

What each part means

- **#1** `---` : YAML document start marker (always start playbooks with it).
- **#2** `name:` : A human-readable name for this **PLAY**. Shown in ansible-playbook output.
- **#3** `hosts:` : Which inventory group(s) this play runs against (here: `webservers`).
- **#4** `become:` : Run all tasks in this play with privilege escalation (sudo).
- **#5** `tasks:` : The list of tasks to execute, in order.
- **#6** task `name` : A label for this individual task — shown in output.
- **#7** `ansible.builtin.dnf` : The fully qualified module name being called.
- **#8** `name: httpd / state: present` : Module arguments as YAML key:value pairs.
- **#18** Second task — starts and enables httpd using the `service` module.

Indentation is 2 spaces per level. Never use tabs in YAML. Mis-indentation is the #1 cause of playbook errors.

Understanding YAML Basics

The 5 rules that cover 90% of Ansible YAML

{} 1. Key-Value Pairs

A key followed by a colon and a value:

```
name: httpd
state: present
become: true
```

2. Indentation (2 spaces)

Nesting = parent/child relationship. Always 2 spaces, never tabs:

```
tasks:
  - name: Install httpd
    ansible.builtin.dnf:
      name: httpd
```

3. Lists (- item)

A list is a sequence of items, each starting with a dash:

```
webservers:
  - web01
  - web02
  - web03
```

Tт 4. Strings

Usually unquoted; quote if they contain special characters:

```
name: httpd
message: "Welcome to Ansible"
path: '/opt/ansible-lab'
```

5. File extension

Ansible playbooks use .yml or .yaml extension:

```
site.yml, install_httpd.yaml, inventory # no ext for inventory
```

␣ 6. Why spacing matters

YAML is whitespace-sensitive. The two below are DIFFERENT:

```
tasks:              # tasks is a key with a list
  - name: foo

tasks : foo         # trailing space ⇒ string value
```

First Ansible Playbook

A minimal playbook that checks uptime

site.yml — your first playbook

```yaml
---
- name: My First Playbook
  hosts: webservers
  tasks:
    - name: Check system uptime
      ansible.builtin.command:
        cmd: uptime
```

Line-by-line explanation

`---`	YAML document start marker (always first line)
`name: My First Playbook`	Name of this play — shown in playbook output
`hosts: webservers`	Run on the 'webservers' inventory group
`tasks:`	Begins the task list
`- name: Check uptime`	First (and only) task — its label
`ansible.builtin.command:`	Fully qualified module name (command module)
`cmd: uptime`	Argument: the actual Linux command to run

Run it

```
$ ansible-playbook -i inventory site.yml

PLAY [My First Playbook] ********************
```

Creating a Directory

Use ansible.builtin.file with state: directory

create_dir.yml

```yaml
---
- name: Create a lab working directory
  hosts: webservers
  become: true
  tasks:
    - name: Ensure /opt/ansible-lab exists
      ansible.builtin.file:
        path: /opt/ansible-lab
        state: directory
        mode: '0755'
```

What this playbook does

`become: true` — the directory is under /opt, which requires root.

`ansible.builtin.file` — the module that manages files and directories.

`path:` — the absolute path to create.

`state: directory` — tells Ansible "this MUST be a directory". If it doesn't exist, Ansible creates it. If it already exists, Ansible leaves it (idempotent).

`mode: '0755'` — set the permissions to rwxr-xr-x. Always quote the mode value in YAML.

Expected output

```
$ ansible-playbook -i inventory create_dir.yml

PLAY [Create a lab working directory] ******
TASK [Ensure /opt/ansible-lab exists] ******
changed: [192.168.1.10]
changed: [192.168.1.11]

PLAY RECAP *********************************
192.168.1.10 : ok=2  changed=1  unreachable=0  failed=0
```

Creating a File

Use ansible.builtin.file with state: touch

create_file.yml

```yaml
---
- name: Create an empty marker file
  hosts: webservers
  become: true
  tasks:
    - name: Ensure /opt/ansible-lab/student.txt exists
      ansible.builtin.file:
        path: /opt/ansible-lab/student.txt
        state: touch
        mode: '0644'
```

Module arguments explained

path: — the full path of the file to create.

state: touch — if the file does not exist, create an empty one. If it exists, update its modification time (like the Linux `touch` command).

mode: '0644' — file readable by everyone, writable only by owner. Quote the value so YAML does not misinterpret the leading 0 as an octal literal.

Verify on a managed host

```
# SSH in (or use an ad-hoc command) to verify:
$ ansible webservers -m command -a "ls -l /opt/ansible-lab/"
192.168.1.10 | CHANGED | rc=0 >>
-rw-r--r-- 1 root root 0 Nov  2 10:15 student.txt
```

state: directory → create a folder

state: touch → create an empty file (or update mtime)

state: file → default; expects file to already exist (used with mode/owner changes)

state: absent → remove the file/directory

Copying Files

Use ansible.builtin.copy with content or src

copy_file.yml

```yaml
---
- name: Copy a welcome message
  hosts: webservers
  become: true
  tasks:
    - name: Create welcome.txt with content
      ansible.builtin.copy:
        content: "Welcome to Ansible Training\n"
        dest: /opt/ansible-lab/welcome.txt
        mode: '0644'
```

src vs content vs dest

content: — literal text written to `dest` on the managed node. Great for small files and config snippets.

src: — a path on the CONTROL NODE to a file copied to `dest` on the managed node. Use for existing files in your project folder.

dest: — the absolute path on the managed node where the file should end up. Always required.

mode: — file permissions on the destination.

Alternative — copy a file from control node

```yaml
# Alternative: copy a file from your control node
- name: Copy index.html from local file
  ansible.builtin.copy:
    src: files/index.html
    dest: /var/www/html/index.html
    mode: '0644'
```

Idempotency: if the destination file already has the exact content, Ansible reports `ok` (not `changed`) and does not rewrite it. This avoids unnecessary file copies and service reloads.

Installing Packages

Use ansible.builtin.dnf to install RPM packages

install_httpd.yml

```yaml
---
- name: Install the Apache HTTP Server
  hosts: webservers
  become: true
  tasks:
    - name: Install httpd using dnf
      ansible.builtin.dnf:
        name: httpd
        state: present
```

dnf module — state options

state	Behavior
`state: present`	Package is installed (no-op if already present)
`state: latest`	Package is installed AND upgraded to newest version
`state: absent`	Package is removed if installed

- **`become: true`** is REQUIRED for dnf — installing packages needs root.
- Use a list to install several packages in one task:

```
name:
  - httpd
  - httpd-tools
  - firewalld
```

- The dnf module targets RHEL / AlmaLinux / Rocky / CentOS / Fedora. For Debian/Ubuntu use **`ansible.builtin.apt`**.

Verify

```
$ ansible webservers -i inventory -m command -a "rpm -q httpd"
192.168.1.10 | SUCCESS | rc=0 >>
httpd-2.4.57-5.el9.x86_64
192.168.1.11 | SUCCESS | rc=0 >>
```

Managing Services

Use ansible.builtin.service to start and enable httpd

start_httpd.yml

```yaml
---
- name: Start and enable Apache
  hosts: webservers
  become: true
  tasks:
    - name: Ensure httpd is started and enabled on boot
      ansible.builtin.service:
        name: httpd
        state: started
        enabled: true
```

service module — state options

state	Behavior
`state: started`	Service is running (starts it if stopped)
`state: stopped`	Service is not running (stops it if running)
`state: restarted`	Service is restarted (always reports changed)
`state: reloaded`	Service config is reloaded without full restart

- **`enabled: true`** — service starts automatically on boot (like `systemctl enable`).
- **`enabled: false`** — service does NOT start on boot.
- **`become: true`** is REQUIRED — managing services needs root.
- The service module is a wrapper that calls the right init system (systemd on modern Linux) under the hood.

Verify

```
$ ansible webservers -i inventory -b -m command -a "systemctl status httpd"
... active (running) since ...
... enabled; preset: enabled)
```

Running a Playbook

ansible-playbook and what its output means

Run with default inventory

```
$ ansible-playbook site.yml
```

Run with explicit inventory

```
$ ansible-playbook -i inventory site.yml
```

- The first form relies on `ansible.cfg` to set the inventory path.
- The second form overrides it with the `-i` flag — useful when you have multiple inventories (dev/stage/prod).

Sample output

Read the output

- **ok=N** — tasks that ran successfully and made no changes.
- **changed=N** — tasks that ran successfully AND changed the system.
- **unreachable=0** — hosts Ansible could not contact over SSH.
- **failed=0** — tasks that errored out (non-zero exit).

Useful flags:

`--syntax-check` — validate YAML without running
`--check` — dry run: show what WOULD change
`-v / -vv / -vvv` — increase verbosity
`--limit web01` — run only against a single host
`--tags install` — run only tasks tagged 'install'

What are Variables?

Names that store values — reused across your playbook

- A variable is a **named container** that holds a value you can reuse in your playbook.
- Real-world analogy: a variable is like a label on a box. You write "username" on the box; the box currently holds "jooli".
- In Ansible, once you define `username: jooli`, you can refer to `{{ username }}` anywhere in the playbook and Ansible substitutes "jooli" at runtime.
- Variables let you write **ONE playbook that works for many environments** — dev, test, prod — just by changing the variable values.
- They make automation flexible: instead of hard-coding a path, package name, or user, you store it once and reference it everywhere.
- If a value needs to change, you change it in **ONE place** — the variable definition — not in 20 tasks.

A simple real-world analogy

> *You are filling out 100 forms that all need the same student's name: **'Jooli'**.*
>
> *Without a variable: you write 'Jooli' 100 times. If the name changes, you re-write 100 forms.*
> *With a variable: you set `student_name: Jooli` once, and the form template says `{{ student_name }}`. Change the variable → all 100 forms update.*

```yaml
vars:
  student_name: jooli

tasks:
  - name: Greet the student
    ansible.builtin.debug:
      msg: "Hello, {{ student_name }}!"
```

Why Do We Use Variables?

Flexibility, reuse, and easy maintenance

Without variables (hard-coded)

- Every value (path, user, package) is typed directly into each task.
- The same value appears in many places — change it once, you must change it everywhere.
- The playbook only works for ONE specific environment.
- Sharing the playbook with another team requires editing every task.
- A typo in one task creates silent inconsistencies between servers.

```
- name: Create directory for jooli
  ansible.builtin.file:
    path: /opt/jooli        # ← hard-coded
    state: directory
- name: Create user jooli
  ansible.builtin.user:
    name: jooli             # ← hard-coded again
    state: present
```

With variables (flexible)

- Define each value once at the top of the playbook.
- Reference it everywhere with `{{ var_name }}`.
- Same playbook works for dev, stage, prod — just swap the variable file.
- Easy to review: all tunable values are in one place.
- Easy to share: another team supplies their own variable values.

```
vars:
  username: jooli
tasks:
  - name: Create directory for {{ username }}
    ansible.builtin.file:
      path: /opt/{{ username }}
      state: directory
  - name: Create user {{ username }}
    ansible.builtin.user:
      name: "{{ username }}"
      state: present
```

"Variables turn a one-environment script into a reusable template."

Defining Variables in a Playbook

vars: block + ansible.builtin.debug

vars_demo.yml

```yaml
---
- name: Demonstrate Variables
  hosts: webservers
  vars:
    username: jooli
  tasks:
    - name: Display username
      ansible.builtin.debug:
        var: username
```

What is happening here?

01 **vars:** — a top-level key inside the play that defines variables for THIS play.

02 **username: jooli** — a variable named `username` holding the value `jooli`.

03 **ansible.builtin.debug** — a module that prints values during playbook execution. Perfect for teaching and debugging.

04 **var: username** — tells debug to print the value of the `username` variable.

Expected output

```
$ ansible-playbook -i inventory vars_demo.yml

PLAY [Demonstrate Variables] ***************
TASK [Display username] ********************
ok: [192.168.1.10] => {
    "username": "jooli"
}
```

Using Variables in Tasks

Variable interpolation with {{ variable_name }}

vars_in_tasks.yml

```yaml
---
- name: Create a user and a home directory using variables
  hosts: webservers
  become: true
  vars:
    username: jooli
    user_home: /opt/ansible-lab/jooli
  tasks:
    - name: Create the user {{ username }}
      ansible.builtin.user:
        name: "{{ username }}"
        state: present
    - name: Ensure {{ user_home }} exists
      ansible.builtin.file:
        path: "{{ user_home }}"
        state: directory
        mode: '0755'
```

Variable interpolation rules

- **Syntax:** `{{ variable_name }}` — double curly braces with the variable name inside.
- Spaces inside the braces are optional but recommended: `{{ username }}` reads better than `{{username}}`.
- **IMPORTANT YAML rule:** when `{{ }}` appears as the START of a YAML value, ALWAYS quote the entire value: `name: "{{ username }}"`. Without quotes, YAML can mis-parse `{{` as a flow mapping.
- You can use variables inside strings: `msg: "Hello, {{ username }} — your home is {{ user_home }}"`.
- Variables are evaluated at runtime, per host, when the task executes.

What happens at runtime

```yaml
# Ansible sees this when running on webservers:
- name: Create the user jooli
  ansible.builtin.user:
    name: jooli
    state: present
- name: Ensure /opt/ansible-lab/jooli exists
  ansible.builtin.file:
    path: /opt/ansible-lab/jooli
```

Different Ways to Define Variables

Four common sources, ordered by precedence

1. Playbook variables

Defined inside the play with a `vars:` block.

Scope: this play only.

```
vars:
  username: jooli
```

2. Separate variable files

Define vars in a YAML file and include it with `vars_files:`.

Scope: this play.

```
vars_files:
  - vars/users.yml

# vars/users.yml
username: jooli
```

3. Inventory variables

Defined per host or per group in the inventory or in group_vars/host_vars directories.

Scope: the host or group they belong to.

```
# in inventory
[webservers]
web01 username=jooli

# group_vars/webservers.yml
username: jooli
```

4. Command-line variables

Pass values at runtime with `--extra-vars` (or `-e`).

Scope: overrides everything else (highest precedence).

```
$ ansible-playbook site.yml -e "username=jooli"
```

Precedence (low → high)

inventory defaults < inventory group_vars < inventory host_vars < playbook vars < vars_files < **--extra-vars (-e)**

When the same variable is defined in multiple places, the HIGHER precedence wins. This is why `-e` is great for one-off overrides without editing files.

What are Ansible Facts?

System information Ansible collects automatically

- Facts are pieces of information about each managed host that Ansible **collects automatically** at the start of every play.
- Think of facts as Ansible asking each server: "Who are you? What OS are you running? How much RAM do you have? What is your IP?"
- Facts are stored in a special variable called `ansible_facts` and are available to every task in the play.
- You do **NOT** define facts yourself — Ansible discovers them by running Python code on the managed node.
- Facts let you write conditional playbooks: "if the OS is RHEL, install with dnf; if Debian, install with apt."

Variables vs Facts

Concept	Origin
Variables	YOU define them. You choose the name and the value.
Facts	ANSIBLE collects them. The names are predefined (`ansible_facts.*`).

```
# Variable - you wrote this:
username: jooli

# Fact - Ansible filled this in for you:
ansible_facts.hostname: "web01"
ansible_facts.os_family: "RedHat"
```

What Information Do Facts Contain?

A snapshot of every managed host

`ansible_facts.hostname`
The short hostname of the server (e.g. "web01").

`ansible_facts.fqdn`
The fully qualified domain name (e.g. "web01.example.com").

`ansible_facts.default_ipv4.address`
The default IPv4 address Ansible will use to reach the host.

`ansible_facts.distribution`
The Linux distribution name (e.g. "AlmaLinux", "Ubuntu").

`ansible_facts.memory_mb.real.total`
Total RAM in megabytes.

`ansible_facts.processor_vcpus`
Number of logical CPUs available.

And many more

Facts also expose: kernel version, architecture, mounted filesystems, network interfaces (with MAC and IPs), DNS servers, default gateway, OS major version, virtualization type, and more. Run `ansible all -m setup` to see everything Ansible knows about your hosts.

Gathering Facts

The setup module and automatic gathering

Gather facts ad-hoc

```
$ ansible all -i inventory -m setup

192.168.1.10 | SUCCESS => {
    "ansible_facts": {
        "ansible_all_ipv4_addresses": ["192.168.1.10"],
        "ansible_architecture": "x86_64",
        "ansible_distribution": "AlmaLinux",
        "ansible_distribution_major_version": "9",
        "ansible_fqdn": "web01.example.com",
        "ansible_hostname": "web01",
        "ansible_memory_mb": {
            "real": { "total": 2048, "used": 412, "free": 1636 }
        },
        "ansible_os_family": "RedHat",
        "ansible_processor_vcpus": 2,
        ...
    },
    "changed": false
}
```

Two ways facts get gathered

1. Manually with the setup module

The command on the left runs `ansible.builtin.setup` on every host and prints all facts as JSON. Use this for exploration and debugging.

2. Automatically in every play

Every play gathers facts BEFORE running its tasks — unless you tell it not to. The facts are then available as `ansible_facts.*` in every task.

Disable gathering when not needed

```
---
- name: Fast playbook - no fact gathering
  hosts: webservers
  gather_facts: false
  tasks:
    - name: Just restart a service
      ansible.builtin.service:
        name: httpd
        state: restarted
```

Displaying Facts Using Debug

Print facts with no manual variables needed

Output (truncated)

```
$ ansible-playbook -i inventory show_facts.yml

TASK [Display system information] ***********
ok: [192.168.1.10] => {
    "ansible_facts": {
        "ansible_distribution": "AlmaLinux",
        "ansible_distribution_major_version": "9",
        "ansible_fqdn": "web01.example.com",
        "ansible_hostname": "web01",
        "ansible_memory_mb": {
            "real": { "total": 2048 }
        },
        "ansible_os_family": "RedHat",
        "ansible_processor_vcpus": 2
    }
}
```

Why no manual variable is required

- Ansible runs the built-in **setup** module automatically at the start of every play.
- The collected information is stored in **ansible_facts** and is available to every task in the play.
- You did not have to define **ansible_facts** yourself — Ansible populated it for you.
- This is what makes facts different from variables: **facts are discovered, variables are declared**.

Practical Facts Examples

Access specific fact values with dot notation

1. Display hostname

```
---
- name: Show hostname
  hosts: all
  tasks:
    - name: Print hostname
      ansible.builtin.debug:
        var: ansible_facts.hostname
```

```
ok: [web01] => { "ansible_facts.hostname": "web01" }
```

2. Display OS distribution

```
---
- name: Show OS
  hosts: all
  tasks:
    - name: Print distribution
      ansible.builtin.debug:
        var: ansible_facts.distribution
```

```
ok: [web01] => { "ansible_facts.distribution": "AlmaLinux" }
```

3. Display default IPv4 address

```
---
- name: Show IP
  hosts: all
  tasks:
    - name: Print default IPv4
      ansible.builtin.debug:
        var: ansible_facts.default_ipv4.address
```

```
ok: [web01] => { "ansible_facts.default_ipv4.address": "192.168.1.10" }
```

4. Display total memory

```
---
- name: Show RAM
  hosts: all
  tasks:
    - name: Print total memory (MB)
      ansible.builtin.debug:
        var: ansible_facts.memory_mb.real.total
```

```
ok: [web01] => { "ansible_facts.memory_mb.real.total": 2048 }
```

How to access nested fact values

- Facts are stored as nested dictionaries. Use dot notation to drill down: `ansible_facts.default_ipv4.address`.
- For dictionary keys with dots or special characters in the name, use bracket notation: `ansible_facts['default_ipv4']['address']`.
- Combine facts with variables in any task: `msg: "Host {{ ansible_facts.hostname }} runs {{ ansible_facts.distribution }} and has {{ ansible_facts.memory_mb.real.total }} MB RAM."`

What is a Loop?

Run the same task many times with different values

- A loop is a way to execute **ONE task multiple times**, each time with a different value from a list.
- Instead of writing five near-identical tasks to create five users, you write **ONE task** and give it a list of five names.
- Ansible iterates over the list, running the task once per item.
- During each iteration, the current list value is available as the magic variable `item`.
- Loops make playbooks shorter, cleaner, and far easier to maintain.
- Loops are the Ansible equivalent of a `for` loop in a programming language.

Repeated execution — the idea

> *Pretend you need to create 5 files: file1.txt, file2.txt, file3.txt, file4.txt, file5.txt.*
>
> *Without a loop:* **5 separate tasks**, *each almost identical.*
> *With a loop:* **1 task + a list of 5 names**.
>
> *Ansible handles the iteration; you just supply the list.*

```yaml
loop:
  - file1.txt
  - file2.txt
  - file3.txt

# Ansible runs the task 3 times. Each time, `item` holds one name.
```

Why Do We Use Loops?

Five tasks become one

✕ Without a loop — 5 separate tasks

no_loop.yml

```yaml
---
- name: Create five users the hard way
  hosts: webservers
  become: true
  tasks:
    - name: Create alice
      ansible.builtin.user: { name: alice }
    - name: Create bob
      ansible.builtin.user: { name: bob }
    - name: Create carol
      ansible.builtin.user: { name: carol }
    - name: Create dave
      ansible.builtin.user: { name: dave }
    - name: Create eve
      ansible.builtin.user: { name: eve }
```

✓ With a loop — 1 task + a list

with_loop.yml

```yaml
---
- name: Create five users the smart way
  hosts: webservers
  become: true
  vars:
    users:
      - alice
      - bob
      - carol
      - dave
      - eve
  tasks:
    - name: Create all users in one task
      ansible.builtin.user:
        name: "{{ item }}"
        state: present
      loop: "{{ users }}"
```

Loop Syntax

The loop keyword and the item variable

loop_syntax.yml

```yaml
---
- name: Loop syntax demo
  hosts: webservers
  tasks:
    - name: Print each name
      ansible.builtin.debug:
        msg: "Hello, {{ item }}"
      loop:
        - alice
        - bob
        - carol
```

🛠 The two pieces of every loop

`loop:`

A task-level keyword that takes a list of values. Ansible runs the task once per item in the list.

`item`

A magic variable Ansible creates automatically during each iteration. It holds the current list value. You reference it as `{{ item }}`.

Modern syntax: The keyword is `loop:` (modern, recommended). Older Ansible code may use `with_items:` — it still works but `loop` is preferred in new playbooks.

Output

```
$ ansible-playbook loop_syntax.yml

TASK [Print each name] **********************
ok: [192.168.1.10] => (item=alice) => {
    "msg": "Hello, alice"
}
ok: [192.168.1.10] => (item=bob) => {
    "msg": "Hello, bob"
```

Practical Loop — Creating Multiple Files

One task, three files in /opt/ansible-lab

loop_files.yml

```yaml
---
- name: Create multiple files with a loop
  hosts: webservers
  become: true
  tasks:
    - name: Ensure lab files exist
      ansible.builtin.file:
        path: "/opt/ansible-lab/{{ item }}"
        state: touch
        mode: '0644'
      loop:
        - file1.txt
        - file2.txt
        - file3.txt
```

How Ansible executes this

01 Ansible starts the task **`Ensure lab files exist`**.

02 It reads the **`loop:`** list: [file1.txt, file2.txt, file3.txt].

03 Iteration 1: **`item`** = "file1.txt" → creates /opt/ansible-lab/file1.txt

04 Iteration 2: **`item`** = "file2.txt" → creates /opt/ansible-lab/file2.txt

05 Iteration 3: **`item`** = "file3.txt" → creates /opt/ansible-lab/file3.txt

06 Ansible reports one result line per iteration.

Verify

```
$ ansible webservers -i inventory -m command -a "ls /opt/ansible-lab/"

192.168.1.10 | CHANGED | rc=0 >>
file1.txt
file2.txt
file3.txt
```

Practical Loop — Creating Multiple Users

Loop over a list of usernames

loop_users.yml

```yaml
---
- name: Create multiple Linux users
  hosts: webservers
  become: true
  vars:
    usernames:
      - alice
      - bob
      - carol
      - dave
  tasks:
    - name: Create each user account
      ansible.builtin.user:
        name: "{{ item }}"
        state: present
        shell: /bin/bash
      loop: "{{ usernames }}"
```

Line-by-line

- `vars:` block defines a list variable `usernames` with 4 entries.
- `become: true` — creating users requires root.
- `ansible.builtin.user` — the module that manages Linux user accounts.
- `name: "{{ item }}"` — each iteration creates the user whose name is in `item`.
- `state: present` — ensure the user exists (creates if missing, no-op if already present).
- `shell: /bin/bash` — set the user's login shell.
- `loop: "{{ usernames }}"` — iterate over the list defined in `vars`.

run output

```
$ ansible-playbook -i inventory loop_users.yml

TASK [Create each user account] *************
changed: [192.168.1.10] => (item=alice)
changed: [192.168.1.10] => (item=bob)
changed: [192.168.1.10] => (item=carol)
changed: [192.168.1.10] => (item=dave)
changed: [192.168.1.11] => (item=alice)
...
```

Going further: loop over a list of DICTIONARIES to set different shells or UIDs per user:

What are Handlers?

MODULE 08 / 11

Tasks that run only when something changed

- A handler is a special kind of task that runs **ONLY** when it is explicitly notified by another task.
- Handlers are typically used for actions that should happen **ONLY after a configuration change** — like restarting a service.
- Handlers look like normal tasks (same module, same arguments) but they live under the `handlers:` block instead of `tasks:`.
- A handler does **NOT** run on every playbook execution — only when a task that notifies it reports `changed: true`.
- This avoids unnecessary service restarts: if a config file did not change, the service does not need to be reloaded.

Handler vs Normal Task

Type	Behavior
Normal task	Runs every time the playbook runs (subject to idempotency).
Handler	Runs **ONLY** when notified by a task that reported `changed`.

anatomy of notify + handler

```
tasks:
  - name: Copy httpd config
    ansible.builtin.copy: ...
    notify: Restart Apache      # ← notifies the handler

handlers:
  - name: Restart Apache        # ← the handler itself
    ansible.builtin.service:
      name: httpd
      state: restarted
```

Why Do We Use Handlers?

Efficiency: act only when needed

01

Run actions only when notified

A handler waits quietly until a task explicitly calls it with `notify:`. If no task notifies it, the handler is skipped — no work, no log entry, no restart.

02

Avoid unnecessary service restarts

Restarting a service drops active connections, causes a brief outage, and reloads state. If the config file did not change, restarting is wasted work and a small disruption. Handlers skip the restart when nothing changed.

03

Improve automation efficiency

Across many hosts, skipping needless restarts saves time, reduces load, and avoids cascading effects (like all web servers restarting simultaneously on every playbook run).

✕ WITHOUT a handler

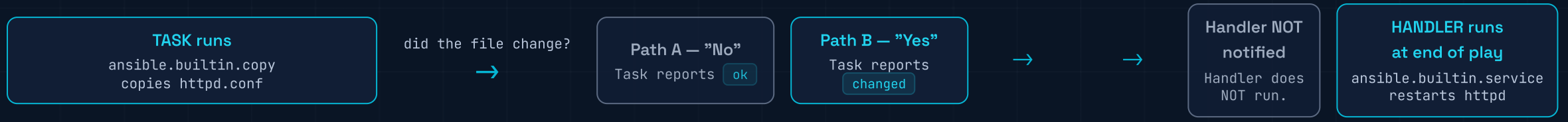
```
- Task: copy httpd.conf
- Task: restart httpd ← runs every time, even if httpd.conf was unchanged.
```

✓ WITH a handler

```
- Task: copy httpd.conf → notify: restart httpd
- Handler: restart httpd ← runs ONLY if the copy changed something.
```

Understanding notify and handlers

How a task triggers a handler



The notify / handler relationship

01 A **TASK** declares `notify: <handler-name>` in its body.

02 The task executes. If it reports `changed: true`, the named handler is queued.

03 If the task reports `ok` (no changes), the handler is **NOT** queued.

04 At the **END of the play** (after all tasks), each queued handler runs exactly once — even if multiple tasks notified it.

05 Handlers run in the **ORDER THEY ARE DEFINED** in the `handlers:` block, not in the order they were notified.

> *Handlers always run at the END of a play, not at the moment they are notified. This batches restarts and avoids repeated restarts within one play.*

Practical Handler Example

Copy Apache config → restart on change

httpd_with_handler.yml

```yaml
---
- name: Configure Apache and restart on change
  hosts: webservers
  become: true
  tasks:
    - name: Copy custom httpd config
      ansible.builtin.copy:
        content: |
          ServerRoot "/etc/httpd"
          Listen 80
          ServerAdmin admin@example.com
        dest: /etc/httpd/conf/httpd.conf
        mode: '0644'
      notify: Restart Apache

    - name: Ensure httpd is running
      ansible.builtin.service:
        name: httpd
        state: started
        enabled: true

  handlers:
    - name: Restart Apache
      ansible.builtin.service:
        name: httpd
        state: restarted
```

Execution flow

01 Play starts on `webservers`. Ansible gathers facts (default behavior).

02 Task 1: `Copy custom httpd config` writes a new httpd.conf.
- If the content differs from the existing file → `changed`, queues handler `Restart Apache`.
- If the content is identical → `ok`, handler is NOT queued.

03 Task 2: `Ensure httpd is running` — starts httpd if it is not already running. (Normal task, not a handler.)

04 End of play: queued handlers run. If `Restart Apache` was queued, httpd is restarted NOW.

05 Ansible prints the PLAY RECAP.

Scenario A — first run

```
TASK [Copy custom httpd config] ***
changed: [192.168.1.10]
TASK [Ensure httpd is running] ****
ok: [192.168.1.10]
RUNNING HANDLER [Restart Apache] *
changed: [192.168.1.10]
```

Scenario B — second run

```
TASK [Copy custom httpd config] ***
ok: [192.168.1.10]          # no cha
TASK [Ensure httpd is running] ****
ok: [192.168.1.10]
# HANDLER NOT RUN - httpd was
# NOT restarted.
```

What is Ansible Vault?

MODULE 09 / 11

Encrypt sensitive files with a password

- Ansible Vault is a built-in feature that **encrypts files** (or individual variables) so their contents cannot be read without a password.
- It uses **AES-256 symmetric encryption** — the same password both encrypts and decrypts the file.
- Encrypted files look like gibberish to anyone without the password — safe to commit to git, safe to email, safe to leave on a shared file server.
- Ansible can decrypt vault-encrypted files **transparently** at playbook runtime — you just provide the password.
- Typical use: encrypt a YAML file containing database passwords, API tokens, or SSH keys, and include it in your playbooks as a variables file.

⚠ Why encryption matters

Imagine your playbook contains:

```
db_password: SuperSecret123
```

If you commit that to git, anyone with read access to the repo — now and forever — sees the password. If a laptop is stolen, if a backup leaks, if a contractor clones the repo: the password is exposed.

With Ansible Vault, the same file at rest looks like:

```
$ANSIBLE_VAULT;1.1;AES256 3631356430383430383564 38... (many lines of ciphertext)
```

Only someone with the vault password can decrypt it. Safe to commit, safe to share.

What is Secrets Management?

Protecting the data that should never be public

- A **secret** is any piece of data that grants access or proves identity — and must **NOT** be exposed to unauthorized people.
- Storing plain-text passwords, API keys, or certificates inside playbooks is a **serious security risk**.
- Plain text secrets in git are a **permanent liability**: even after deletion, they remain in git history.
- **Secrets management** is the practice of storing, accessing, rotating, and auditing secrets securely.
- Anyone with read access to your automation repo can read **every** secret in it.

Passwords

Database passwords, application passwords, default account passwords.

```
db_password: hunter2
```

SSH keys

Private SSH keys used by Ansible to connect to managed nodes.

```
~/.ssh/id_rsa contents stored as a variable.
```

API tokens

Cloud provider tokens (AWS access keys), CI/CD tokens, SaaS API keys.

```
aws_access_key_id, aws_secret_access_key
```

Database credentials

Connection strings with embedded credentials.

```
postgresql://appuser:secret@db01/appdb
```

Certificates & TLS keys

SSL/TLS private keys used by web servers and services.

```
/etc/pki/tls/private/server.key
```

Sensitive config values

Any config file containing a password or token that Ansible deploys.

```
properties file with a mail relay password
```

RULE: If you would not be comfortable printing it on a poster in the office, it belongs in **Ansible Vault** — not in plain text.

Ansible Vault for Secrets Management

Encrypt, store, and use secrets safely

- Ansible Vault turns plain-text variable files into **encrypted files** that are useless without the password.
- Encrypted files can be safely committed to git, shared via email, or stored on shared drives.
- At playbook runtime, Ansible decrypts the file **transparently** — you only need to provide the vault password once.
- The **same vault password** is used to encrypt AND decrypt — keep it safe, and share it only with people who need to run the automation.
- Vault works on whole files (most common) or on **individual variable values** inside a larger file.
- Encrypted variables are decrypted **on-demand**, only when Ansible needs their value.

⇄ The vault workflow

01 You create an encrypted file (e.g. `secrets.yml`) with `ansible-vault create`.

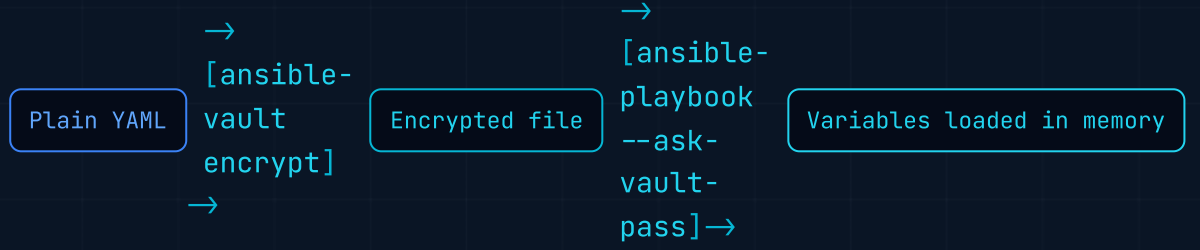
02 You commit the **ENCRYPTED** file to git — it looks like ciphertext, safe to share.

03 You reference the file from your playbook via `vars_files:`.

04 At runtime, you provide the vault password (`--ask-vault-pass` or via a password file).

05 Ansible decrypts `secrets.yml` in memory, loads the variables, and uses them like any other variable.

06 The plain-text value **never touches disk** unencrypted (except in memory during the run).



Creating an Encrypted File

ansible-vault create secrets.yml

Create a brand-new encrypted file

```
$ ansible-vault create secrets.yml
New Vault password:
Confirm New Vault password:
# (your editor opens - type your variables, then save and quit)

# The file is saved as encrypted ciphertext:
$ cat secrets.yml
$ANSIBLE_VAULT;1.1;AES256
396263333938343735373734383038...
363135643038343038356438336239...
613839343262303239373131353031...
...
```

What just happened

01 `ansible-vault create secrets.yml` opens an interactive prompt for a new vault password.

02 You type the password **twice** (for confirmation).

03 Ansible opens your default text editor (`$EDITOR` — usually vi or nano).

04 Inside the editor, you type your variables as normal YAML:

```
db_password: SuperSecret123
api_token: abc123XYZ
```

05 When you save and quit, Ansible encrypts the buffer with your password and writes the ciphertext to `secrets.yml`.

06 The file on disk is now encrypted — `cat` shows ciphertext, not your real values.

The vault password is NOT stored anywhere by Ansible. Lose it = lose your secrets. Store it in a password manager or a real secrets backend.

Encrypting and Decrypting Files

Three commands you will use constantly

Encrypt an existing plain-text file

```
# You already have secrets.yml in
# plain text. Encrypt it in place:
$ ansible-vault encrypt secrets.yml
New Vault password:
Confirm New Vault password:
Encryption successful
```

Decrypt an encrypted file back to plain text

```
# Decrypt in place (now plain text
# again - careful!):
$ ansible-vault decrypt secrets.yml
Vault password:
Decryption successful
```

View without saving plain text

```
# Safely view the contents without
# writing plain text to disk:
$ ansible-vault view secrets.yml
Vault password:
db_password: SuperSecret123
api_token: abc123XYZ
```

Command	Effect on the file
`ansible-vault create FILE`	Create a new encrypted file (interactive editor).
`ansible-vault encrypt FILE`	Encrypt an existing plain-text file **IN PLACE**.
`ansible-vault decrypt FILE`	Decrypt an encrypted file **IN PLACE** (writes plain text!).
`ansible-vault view FILE`	View decrypted contents **WITHOUT** saving plain text.
`ansible-vault edit FILE`	Open encrypted file in editor; re-encrypts on save.

⚠ **WARNING:** `ansible-vault decrypt` writes **PLAIN TEXT** to disk. Use it only temporarily, then re-encrypt. Prefer `ansible-vault view` or `ansible-vault edit` for day-to-day work.

Editing and Using Vault Files

Edit encrypted files safely; use them in playbooks

Edit an encrypted file safely

```
$ ansible-vault edit secrets.yml
Vault password:
# (editor opens with decrypted content;
#  on save, file is re-encrypted)

# File on disk remains encrypted the whole
# time. Plain text exists only in your
# editor session.
```

Use the encrypted file in a playbook

```
# Run a playbook that includes secrets.yml:
$ ansible-playbook -i inventory site.yml --ask-vault-pass
Vault password:
PLAY [Provision database] ********************
```

How it all fits together

01 `ansible-vault edit secrets.yml` decrypts the file in memory only, opens your editor, and re-encrypts when you save. The plain-text version is never written to disk.

02 Your playbook references the encrypted file via `vars_files:`

```
---
- name: Provision database
  hosts: dbservers
  vars_files:
    - secrets.yml
  tasks:
    - name: Show database password (for demo only)
      ansible.builtin.debug:
        var: db_password
```

03 At runtime you pass `--ask-vault-pass`; Ansible prompts for the vault password, decrypts `secrets.yml` in memory, and makes `db_password` available as a normal variable inside the play.

Best practice — vault password file

```
# Use a vault password file to avoid typing
# the password each run:
$ echo 'myVaultPassword123' > ~/.vault_pass
$ chmod 600 ~/.vault_pass
$ ansible-playbook -i inventory site.yml \
    --vault-password-file ~/.vault_pass

# Or set it permanently in ansible.cfg:
# [defaults]
# vault_password_file = ~/.vault_pass
```

What are Ansible Roles?

MODULE 10 / 11

Structured, reusable automation packages

- A **role** is a way to package Ansible automation into a structured, reusable unit.
- A role bundles **tasks, handlers, variables, files, templates, and defaults** into a standardized directory layout.
- Instead of one giant playbook with hundreds of lines, you split the work into roles: one role for Apache, one for MySQL, one for firewalling.
- A playbook then simply lists **which roles to apply** to which hosts.
- Roles make automation **modular**: build once, use everywhere.
- Roles are how Ansible automation is **shared between teams** and across the community via Ansible Galaxy.

Role vs Playbook

Type	What it is
Playbook	A YAML file with plays and tasks. Good for small, project-specific work.
Role	A packaged directory of tasks/vars/handlers/files. Reusable across projects.

```
# BEFORE: one giant playbook site.yml (500 lines, mixed concerns) # AFTER: split into roles site.yml (10 lines, just lists roles) roles/ apache/ ← install + configure Apache mysql/ ← install + configure MySQL firewall/ ← manage firewalld
```

Why Do We Use Roles?

Organization, reuse, and maintainability

01 **Organize large playbooks**

Split a 500-line playbook into 5 focused roles of 100 lines each. Each role has one job (Apache, MySQL, firewall, users, monitoring).

02 **Separate tasks, variables, and handlers**

Roles enforce a standard directory layout. Tasks live in `tasks/`, handlers in `handlers/`, variables in `vars/` and `defaults/`. No more hunting through one file.

03 **Reuse automation**

A well-written Apache role can be used by the web team, the QA team, and the lab team — all importing the same role with different variables.

04 **Maintain projects easily**

Bug fix in the Apache role? Fix it once in the role, every project that uses the role gets the fix on next run. No copy-paste across dozens of playbooks.

05 **Share automation between projects and teams**

Push the role to git or Ansible Galaxy. Other teams install it with one command. Standards spread naturally.

"Roles are to Ansible what functions are to programming: packages of reusable behavior."

Ansible Role Directory Structure

The standard layout Ansible expects

tree view of an 'apache' role

```
apache/                  # role root - role name = directory name
  tasks/
    main.yml         # the TASKS the role runs (REQUIRED)
  handlers/
    main.yml         # handlers used by the role's tasks
  templates/             # Jinja2 template files (.j2)
  files/                 # static files to copy to managed hosts
  vars/
    main.yml         # role's INTERNAL variables (high precedence)
  defaults/
    main.yml         # role's DEFAULT variable values (low precedence - over
  meta/
    main.yml         # role metadata: author, dependencies, supported platfo
```

What each directory holds

01 `tasks/main.yml` **REQUIRED**
The main list of tasks the role executes. This is the only strictly required file.

02 `handlers/main.yml`
Handlers the role's tasks may notify (e.g. restart Apache).

03 `templates/`
Jinja2 templates that get rendered with variables and copied to managed hosts.

04 `files/`
Static files copied as-is via `ansible.builtin.copy` (no templating).

05 `vars/main.yml`
Variables the role uses internally; high precedence, hard to override.

06 `defaults/main.yml`
Default variable values that callers CAN override; lowest precedence.

07 `meta/main.yml`
Role metadata: author, description, dependencies on other roles, supported OSes.

Creating an Ansible Role

ansible-galaxy role init apache

Scaffold a new role

```
$ ansible-galaxy role init apache
- Role apache was created successfully

# Ansible generated the standard role directory:
$ ls -F apache/
defaults/  files/  handlers/  meta/  README.md  tasks/  templates/  tests/
```

What ansible-galaxy did for you

- Created the `apache/` directory with the standard subdirectories.
- Added a starter `tasks/main.yml`, `handlers/main.yml`, `vars/main.yml`, `defaults/main.yml`, `meta/main.yml` — each containing just `---` and a comment.
- Added a `README.md` describing the role.
- Added a `tests/` directory with a sample inventory and test playbook.

starter tasks/main.yml

```
$ cat apache/tasks/main.yml
---
# tasks file for apache
```

The role is now ready to fill in. Open `apache/tasks/main.yml` and start adding tasks — the same tasks you wrote in slide 28 (install httpd) and slide 29 (start httpd) now live inside the role.

Using a Role in a Playbook

Call a role with the roles: keyword

site.yml — use the apache role

```yaml
---
- name: Set up web servers using the apache role
  hosts: webservers
  become: true
  roles:
    - apache
```

What this playbook does

- Declares a play targeting the `webservers` group.
- Uses `become: true` so all role tasks run with sudo.
- Lists one role to apply: `apache`.
- Ansible looks for the role in `./roles/apache/` (relative to the playbook) or in the configured roles paths.
- Ansible runs every task defined in `apache/tasks/main.yml` against each host in webservers.

project layout

```
$ tree ansible-lab/
ansible-lab/
  inventory
  site.yml              ← the playbook on the left
  roles/
    apache/
      tasks/main.yml    ← contains install + start tasks
      handlers/main.yml ← handler: restart apache
      defaults/main.yml ← default vars (e.g. httpd_port: 80)
```

Override role defaults

```yaml
---
- name: Web servers with custom Apache port
  hosts: webservers
  become: true
  roles:
    - role: apache
      vars:
        httpd_port: 8080     # overrides the role's default
```

What is Ansible Galaxy?

The community marketplace for Ansible content

- Ansible Galaxy (`galaxy.ansible.com`) is the official community hub for sharing Ansible roles and collections.
- It is to Ansible what **npm** is to JavaScript, or **PyPI** is to Python — a public registry of reusable automation.
- Thousands of community-maintained roles exist: Apache, Nginx, MySQL, Docker, PostgreSQL, Redis, Kubernetes, AWS helpers, and many more.
- You search, preview, and install roles directly from the command line with `ansible-galaxy`.
- Production teams often publish their own internal roles to a private Galaxy or to git, then install them with a `requirements.yml` file.
- Using a well-maintained community role can save days of work — but **always review the code** before relying on it in production.

Install a community role

install role from galaxy

```
$ ansible-galaxy role install geerlingguy.apache
- downloading role 'apache', owned by geerlingguy
- downloading role from https://github.com/geerlingguy/ansible-role-apache...
- extracting geerlingguy.apache to ~/.ansible/roles/geerlingguy.apache
- role geerlingguy.apache was installed successfully
```

- `geerlingguy.apache` is a popular, well-maintained role that installs and configures Apache on RHEL/Debian families.
- After installation, use it in any playbook:

```
roles:
- geerlingguy.apache
```

- The role lives in `~/.ansible/roles/` (default roles path) and is available to every playbook on that control node.

Always review a community role's source on GitHub before installing it in production. You are trusting it to run as root on your servers.

Roles and Ansible Galaxy for Reusable Automation

From solo scripts to shared, maintained automation

STAGE 1

Ad-hoc commands

One-off fixes. Operator-dependent. Not reusable.

```
ansible all -m command -a "df -h"
```

→

STAGE 2

Playbooks

Saved as code. Version controlled. Repeatable within one project.

```
site.yml in your team's git repo.
```

→

STAGE 3

Roles on Galaxy

Packaged, documented, versioned. Reusable across teams and the public.

```
geerlingguy.apache installed by hundreds of organizations.
```

How roles + Galaxy help teams

Share

- Publish roles to a private Galaxy or public Galaxy.
- Other teams install with one command.
- Standards spread naturally across the organization.
- Versioning lets consumers pin to a known-good release.

Reuse & Maintain

- A fix in the role benefits every consumer on next run.
- Documentation lives alongside the role (README.md).
- Tests in `tests/` catch regressions before release.
- New team members ramp up faster — they read roles, not 1000-line playbooks.

"Roles + Galaxy turn Ansible from a tool into an ecosystem."

Complete Ansible Automation Workflow

MODULE 11 / 11

How every concept in this course fits together

computer Control Node	→	inventory Inventory	→
settings ansible.cfg	→	terminal Ad-hoc Commands	→
description Playbooks	→	tune Variables	→
memory Facts	→	repeat Loops	→
notifications_active Handlers	→	lock Vault	→
folder_special Roles	→	public Ansible Galaxy	

How the pieces connect

01 From your **CONTROL NODE**, you define which servers to manage in your **INVENTORY** and tune Ansible's behavior in **ansible.cfg**.

02 For quick one-off tasks, you use **AD-HOC COMMANDS**.

03 For repeatable work, you write **PLAYBOOKS**.

04 Inside playbooks you use **VARIABLES** for flexibility, **FACTS** for system data, **LOOPS** for repeated tasks, and **HANDLERS** for event-driven restarts.

05 Sensitive values are protected with **ANSIBLE VAULT**.

06 As playbooks grow, you split them into **ROLES** and share them via **ANSIBLE GALAXY**.

"Each concept is a layer. Stack them to go from a single command to a fully automated, reusable, secure infrastructure."

Important Differences

Side-by-side comparisons to lock in the concepts

Concept A	Concept B	Key difference
Ad-hoc Commands	Playbooks	Ad-hoc: one command, one task, not saved. Playbooks: a YAML file of tasks, version-controlled and repeatable.
Variables	Facts	Variables: **YOU** define and name them. Facts: Ansible collects them automatically (`ansible_facts.*`) at the start of every play.
Tasks	Handlers	Tasks: run every time the playbook runs. Handlers: run only when notified by a task that reported `changed`.
Playbooks	Roles	Playbooks: single YAML files of plays and tasks. Roles: packaged, structured directories (tasks/handlers/vars/...) that playbooks can include and reuse.
Ansible Vault	Plain-text files	Vault: file is encrypted ciphertext at rest; safe to commit. Plain text: anyone with repo access sees the secret forever (even after deletion, due to git history).

Mnemonic

*"**Ad-hoc** is for now. **Playbooks** are for later. **Variables** are yours. **Facts** are discovered. **Tasks** always run. **Handlers** run on change. **Playbooks** scale to **roles**. **Vault** protects what plain text exposes."*

Practical Lab Scenario

Put it all together on three Linux servers

Scenario

You are the administrator for a small environment with three AlmaLinux 9 servers:

```
web01 192.168.1.10
web02 192.168.1.11
db01 192.168.1.20
```

Your manager asks you to fully automate the environment using Ansible:

- verify connectivity,
- create a working directory,
- create lab files,
- install Apache on the web servers,
- start Apache,
- display each server's hostname and OS,
- create multiple Linux users with a loop,
- restart Apache only when its config changes,
- protect the database password using Ansible Vault.

Divide this into beginner-friendly exercises. Each exercise builds on the previous one and exercises one concept from the course.

Exercises

01 **Connectivity** — ad-hoc command `(Module 3)`

```
$ ansible all -i inventory -m ping
```

02 **Create /opt/ansible-lab directory** — playbook + file module `(Module 4)`

```
state: directory
```

03 **Create files file1.txt, file2.txt, file3.txt** — playbook + loop `(Module 7)`

04 **Install httpd on webservers** — playbook + dnf module `(Module 4)`

05 **Start and enable httpd** — playbook + service module `(Module 4)`

06 **Display hostname and OS per host** — playbook + facts + debug `(Module 6)`

07 **Create users alice, bob, carol, dave** — playbook + loop `(Module 7)`

08 **Copy custom httpd.conf and restart Apache only on change** — playbook + handler `(Module 8)`

09 **Store db_password in secrets.yml, use it in a playbook** — ansible-vault `(Module 9)`

End state: *3 servers fully configured, Apache serving on the web tier, secrets protected, every step repeatable by re-running the playbooks.*

Final Revision and Key Takeaways

Your Ansible learning journey — summarized

Architecture

Ansible is **agentless**. A control node pushes commands over SSH to managed Linux nodes. No agent to install; Python is required on managed nodes.

Inventory & ansible.cfg

Inventory lists *WHAT* to manage; **ansible.cfg** controls *HOW* Ansible behaves. Keep both in your project folder under git.

Ad-hoc vs Playbooks

Ad-hoc commands are for quick one-off operations. Playbooks are saved, repeatable YAML. Move a repeated ad-hoc into a playbook.

Variables, Facts, Loops, Handlers

Variables hold YOUR values. **Facts** are gathered by Ansible. **Loops** repeat a task over a list. **Handlers** run only when notified by a changed task.

Vault

Never store passwords or tokens in plain text. Use `ansible-vault` to encrypt secret files; commit the encrypted file, not the plain text.

Roles & Galaxy

Structure large automation into roles. Share roles via Ansible Galaxy. Reuse community roles instead of reinventing.

What to do next

- Reproduce the 9-step practical lab on your own VMs.
- Push your project to a git repo.
- Read the official docs: `docs.ansible.com`
- Explore `ansible-galaxy` for roles relevant to your work.

”Ansible rewards practice. Automate something today — then automate something bigger tomorrow.”

— END OF COURSE —